Container Death Triage â€” telling six lookalikes apart in one step

Revision: 1 **Last modified:** 2026-08-21T17:06:26Z **Audience:** Anyone who has just found a boba container stopped, restarted, or missing, and needs to know *what killed it* before proposing a fix. **Authority:** Derived from BOB-131, where one ticket recorded â€œpodman conmon crashâ€ for two unrelated events, neither of which was a conmon crash. Full evidence: [docs/incidents/2026-08-21-bob131-container-death-triage.md](). **Tool:** [scripts/diagnostics/bob131_container_death_triage.sh]()

Start here

```
S=./scripts/diagnostics/bob131_container_death_triage.sh
$S collect qbittorrent-proxy "2026-08-20 17:55:30" "2026-08-20 17:57:00" >
$S classify /tmp/bundle.txt
```

Timestamps are **host-local** (`journalctl` syntax). The tool is read-only: it never starts, stops, restarts, removes or signals anything.

The decision table

Read **top to bottom** and stop at the first row that matches. The order is not cosmetic â€” the classes overlap in symptoms, and the cheap signals lie.

#	Class	The one signal that settles it	What it means
1	`HOST-POWER-TRANSITION`	`systemd-logind: System is powering down` / a boot boundary in `journalctl --list-boots`	**The container did not fail.** The host went down. Exit code is usually **0**, and *every* container dies at once.
2	`THREAD-LIMIT-EXHAUSTION`	`EAGAIN`, `failed to create new OS thread`, `pthread_create failed: Resource temporarily unavailable`, `Cannot allocate memory` **on fork/clone**	**Â§12.12.** Not an OOM. Compare `ulimit -u` with `ps -L --no-headers -u "$USER" | wc -l`.

#	Class	The one signal that settles it	What it means
3	CGROUP-OOM-KILL	`memory.events: oom_kill` **> 0**, or kernel `Memory cgroup out of memory` naming `libpod-<cid>`	**Â§12.6 containment working as designed.** The container hit *its own* limit. Decode `oom_memcg`: `libpod-â€¦` = the containerâ€™s cap; `user@1000.service` = session OOM.
4	PROCESS-SIGSEGV	`died exit=139` (128+11), **and** a kernel `segfault` / `ANOM_ABEND â€¦ sig=11`	A real crash **inside** the container. Runtime is fine.
5	ORCHESTRATED-STOP	`died exit=137` (128+9) ~10 s after a `restart` / `stop` / `down` event	SIGTERM-timeout SIGKILL. Not a crash, not an OOM. PID 1 was too slow to shut down.
6	CGROUP-MEMORY-CEILING	`memory.events: max` > 0 **with `oom_kill == 0`**	**Pressure, never a cause of death.** The kernel *reclaimed*; it killed nothing.

The three that get confused, and why

`oom_kill > 0` vs `max > 0` â€” a kill vs a squeeze

Both live in the same file and read as â€œmemory troubleâ€. They are not the same event.

```
CID=$(podman inspect qbittorrent-proxy --format '{{.Id}}')
CG=$(find /sys/fs/cgroup -maxdepth 8 -type d -name "libpod-${CID}.scope" |
cat "$CG/memory.events"
```

- **`oom_kill > 0`** â†’ the kernel **killed** something at the cgroup limit.
- **`max > 0`, `oom_kill 0`** â†’ the cgroup **reached** its limit and the kernel reclaimed/swapped instead. Nothing died of it.

Measured on this stack (`mem_limit: 768m` for `download-proxy`):

```
memory.max      805306368     # 768 MiB
memory.peak     805371904     # pinned at the ceiling
memory.events:  max 1584      # 1584 times in ~48 min
                oom_kill 0    # ...and killed nothing
```

That container lives permanently against its ceiling and reports **healthy**. Citing `max 1584` as a cause of death is wrong. It is worth its own ticket as *capacity*; it is not an explanation for a crash.

OOM-kill vs thread exhaustion â€” both say â€œcannot allocateâ€¦, neither is the other

Thread/`RLIMIT_NPROC` exhaustion (Â§12.12) fails `clone(2)` with **`EAGAIN`** while **memory is abundant**. Every memory metric looks healthy while it happens, so it is routinely misfiled as an OOM.

```
ulimit -u; ulimit -Hu                          # the ceiling
ps -L --no-headers -u "$USER" | wc -l          # live threads for this UID
```

Current headroom on this host: limit **65536**, observed peak **1559** (2.4 %). Thread exhaustion is **not** currently plausible here â€” it was, historically, when the limit was 4096.

> **Never** use `pkill -f` / `killall` / `pgrep -f` to investigate: the pattern matches your own command line (Â§11.4.263, Â§12.12). And never signal pgid â‰¤ 1.

â€œconmon crashedâ€¦ â€” almost always a carrier match

`conmon <error>: â€¦` in the log means conmon **reported** a fault. A conmon *crash* would put `conmon` itself in a kernel `segfault` / `ANOM_ABEND` line.

```
# the thing, not a mention of it:
journalctl --since "<window>" | grep -E 'kernel: conmon\[[0-9]+\]: segfaul
```

Empty â‡’ conmon did not crash, whatever the `<error>` lines say. These two are benign races during teardown, not faults:

```
conmon <CID> <error>: Failed to write 137 to exit file: ... No such file o
conmon <CID> <error>: Failed to create container: exit status 1
```

Likewise `conmon <nwarn>: Failed to open cgroups file: /sys/fs/cgroup/memory.events` is ordinary rootless noise â€” **50 290** occurrences in one week here.

Gotchas that will waste your time

- **Timezones disagree.** `podman ps` / some `podman events` records print **`+0300`** for boba containers while the host journal prints **`+0200 CEST`**. Always convert to UTC before correlating: `date -u -d "@<epoch>"`.
- **`podman events --since/--until` may return nothing** for a window that plainly contains events. The journal is authoritative; the tool falls back to it and does not depend on the events stream for its verdict.

- **cgroup files are LIVE.** For a historical window they describe *todayâ€™s* container carrying that name, not the dead one. The bundle labels this.
- **A removed containerâ€™s `podman logs` are gone**, but its stdout survives in the journal because the log driver is `journald`: `journalctl CONTAINER_NAME=qbittorrent-proxy --since â€¦ --until â€¦`. This is how BOB-131â€™s crash was read after the container was long gone.
- **`journalctl | grep` over the full retention takes minutes.** Always bound it with `--since`, and prefix long runs with `nice -n 19 ionice -c 3`.
- **`find` here is `bfs`, not GNU findutils.** Relative `-newermt` forms are rejected â€” they error to stderr and print nothing to stdout.
- **`grep -q` and `awk 'â€¦; exit'` inside a pipeline** close the pipe early and SIGPIPE the writer; under `set -euo pipefail` that aborts your script, and it does so *racily* (Â§11.4.201(12)). Drain instead: `grep -E â€¦ >/dev/null`, `awk 'â€¦' | head -1`.

Boot persistence â€” why a container can be missing with nothing wrong

`restart: unless-stopped` is a within-podman policy. **It does not survive a host power cycle.** On this host, as measured 2026-08-21:

```
podman-restart.service   disabled   (user and system scope)
boba-stack.service       linked, disabled
Linger=yes
```

So after any power-off the boba stack stays down until someone runs `./start.sh`. In BOB-131 that was **14h 06m 43s**. If a container is *absent* rather than *unhealthy*, check the boot boundary before looking for a crash.

Verifying the tool still has teeth

```
./scripts/diagnostics/bob131_container_death_triage.sh --selftest
```

Eight golden fixtures â€” one per class, a negative control that must stay `UNKNOWN`, and a conmon-carrier fixture that must **not** change the verdict. If any mutation of the classifier leaves this green, the check is decorative and must be repaired before it is trusted (Â§1.1, Â§11.4.115(F)).